Problem Set: Article Named-Entity Recognition Service

Audience

Undergraduate data-science students with basic Python, HTTP/API, and introductory natural-language-processing knowledge.

Case background

A news-monitoring team needs a small service that can identify named entities mentioned in an article. The user should be able to enter an article title and body into a simple panel and receive the entities found in the text.

The goal of this problem set is to build a working prototype, not a production-grade information-extraction platform. Students should make their assumptions explicit, choose an appropriate NLP approach, and demonstrate that their service works on both typical and difficult articles.

Learning objectives

By completing this case, students should be able to:

- formulate a data-science problem as a service with clear inputs and outputs;
- preprocess and combine article fields safely;
- apply a named-entity-recognition (NER) model or library;
- distinguish entity text from entity type and character location;
- design a simple user interface and backend endpoint;
- evaluate extraction quality with labeled examples;
- communicate limitations, errors, and ethical considerations.

Assignment

Create a simple web service with a panel that accepts:

1. an article title; and
2. an article body.

When the user submits the form, the service must analyze the combined article and return the named entities it identifies.

Students may use Python with Flask or FastAPI, or another instructor-approved stack. They may use an existing NER model or library, but they must explain which model was selected and why.

Provided data

The working data is in `use_cases/pnm_study_case/sample/`. It consists of semicolon-delimited CSV exports containing online-media articles. The important fields for this case are `title`, `body`, `language`, `entities`, `original_id`, `source_name`, and `date_published`.

Students must load the applicable CSV files from this directory, preserve quoted text correctly, and document the number of rows loaded, duplicate rows found, missing titles or bodies, and language distribution. Do not commit a second copy of the provided articles.

The `entities` column is not automatically a gold-standard NER annotation. Students must inspect its format and provenance. If it is empty, incomplete, or not span-labeled, it may be used for exploration or weak supervision only; it must not be treated as ground truth without validation.

Required behavior

Input

The service must accept a JSON request with this shape:

```
{
  "title": "Example article title",
  "body": "The full article text goes here."
}
```

The title and body must both be strings. The service should reject an empty title and body, malformed JSON, or an input that exceeds a documented maximum length.

Output

For a valid request, return JSON containing the extracted entities. Each entity should include:

- `text`: the exact text span detected;
- `label`: the entity category, such as `PERSON`, `ORG`, `GPE`, `LOC`, `DATE`, or `EVENT`;
- `start`: the zero-based character offset in the analyzed text;
- `end`: the exclusive character offset in the analyzed text.

Example response:

```
{
  "entities": [
    {
      "text": "Ada Lovelace",
      "label": "PERSON",
      "start": 0,
      "end": 12
    },
    {
      "text": "London",
      "label": "GPE",
      "start": 28,
      "end": 34
    }
  ],
  "meta": {
    "entity_count": 2,
    "model": "documented-model-name"
  }
}
```

The service must return a clear error response, preferably with HTTP status `400`, when the request is invalid. The output should be deterministic for the same input and model version.

Functional requirements

The submission must include:

- a browser-accessible panel with fields for title and body;
- a submit action and a visible loading or processing state;
- a readable display of entity text and entity type;
- a backend endpoint that performs the NER analysis;
- input validation on both the panel and backend;
- at least five automated tests or reproducible test cases;
- a README explaining installation, startup, API usage, model choice, and limitations.

The interface may be minimal. A table is sufficient for presenting the returned entities. Students do not need to implement user accounts, persistent storage, article scraping, or deployment to a public cloud. Model training or adaptation using the provided sample data is required for this case.

Suggested workflow

1. Define the entity categories that the chosen model supports.
2. Load and profile the CSV files in `sample/`.
3. Annotate a representative subset and create train, validation, and test splits.
4. Train or adapt the NER pipeline and evaluate it independently from the web interface.
5. Add an API endpoint for title and body input.
6. Add the panel that calls the endpoint and renders its response.

7. Test normal input, empty input, punctuation, repeated entities, numbers and dates, non-ASCII text, and long text.
8. Evaluate the service against the held-out test labels and discuss errors.

Evaluation task

Use the articles in `use_cases/pnm_study_case/sample/` for model development and testing. Prepare a labeled evaluation set of at least 10 short articles or article excerpts selected from those files. The set should contain a mixture of:

- people, organizations, places, dates, and other supported entity types;
- entities appearing more than once;
- ambiguous names and names with punctuation;
- articles containing no named entities;
- at least one example that is challenging for the selected model.

Compare the system output with the reference labels. At minimum, report precision, recall, and F1 score for entity detection. Students should state whether matching is exact-span, partial-span, or entity-type-and-span matching. Include a short error analysis with at least three false positives or false negatives.

Training and testing requirement

This case must include a reproducible train/validation/test workflow. A pretrained NER model may be used as a baseline, but the final submission must include a model or extraction pipeline trained, adapted, or calibrated using examples from the provided sample data. A rules-only solution is not sufficient on its own.

Annotation

Create span-level annotations for selected articles using a documented label scheme. At minimum, label people, organizations, and places; add dates, events, or other categories when appropriate. Store annotations in JSONL, spaCy `DocBin`, BIO/IOB2, or another machine-readable format, retaining the original article identifier and source row reference.

Students may begin with the existing `entities` field, entity dictionaries, or model-assisted suggestions, but a human must review labels used for the final evaluation set. Record annotation guidelines, disagreements, and corrections.

Data split

Split by article identity, not by sentence or row, so text from one article cannot appear in multiple splits. Use a documented deterministic seed. Unless dataset size requires another choice, use 70% training, 15% validation, and 15% held-out test data. Keep the test set untouched until final evaluation. Put duplicates or near-duplicates in the same split or remove them with an explanation.

Model development

Students must:

1. build a baseline using the selected pretrained NER model or a simple non-trained baseline;
2. train or fine-tune the selected model on the training annotations, or adapt its entity-recognition pipeline using the training set;
3. use validation data to select preprocessing, label mappings, hyperparameters, and stopping criteria;
4. evaluate the final untouched model on the test set;
5. save the trained model or provide a reproducible training command; and
6. connect the final trained artifact to the service endpoint.

For small datasets, transfer learning or few-shot adaptation is acceptable; training from scratch is not required. Report document and span counts per split, model settings, training duration, random seed, and class imbalance.

Required reporting

Report results separately for validation and final test data. Include exact-span precision, recall, and F1 overall; per-label metrics; baseline-versus-trained comparison; at least three test-set errors; and a statement about whether test examples influenced annotation or tuning. The service demonstration must use the trained or adapted pipeline evaluated in the report.

Questions to answer

1. What makes a text span a named entity in this problem?
2. Why should title and body be combined before analysis? What information could be lost if they are processed independently?
3. What are the supported labels of the chosen model, and how do they map to the categories shown to users?
4. How does tokenization, punctuation, capitalization, or language affect the result?
5. What is the difference between an entity's text, label, and character offsets?
6. What happens when the same entity appears multiple times with different spelling or capitalization?
7. Which errors are most important for a news-monitoring user, and how could the system be improved?
8. What privacy, copyright, bias, or responsible-use concerns apply to processing article text?

Minimum technical specification

- The service must run locally using documented commands.
- The API must expose one endpoint for entity extraction, such as `POST /extract-entities`.
- The endpoint must return JSON and an appropriate HTTP status code.
- The panel must not crash when the backend returns an error.
- The implementation must not hard-code the entities for the demonstration examples.

- Dependencies and model versions must be recorded.
- Secrets, if any, must be supplied through environment variables and must not be committed.

Submission format

Submit a repository or archive containing:

```
project/
├── README.md
├── requirements.txt or package.json
├── src/
├── tests/
├── data/
│   └── evaluation_examples.json
└── screenshots/
```

The README must include one sample request and response, a short architecture diagram or description, setup commands, and known limitations. Include screenshots only if the instructor requests them.

Assignment scoring

Scoring is based primarily on the quality of three areas:

- **NER implementation:** correctness of the trained or adapted pipeline, entity spans and labels, use of the provided sample data, and integration with the service;
- **Evaluation:** sound annotation and data splitting, reproducible training and testing, appropriate metrics, baseline comparison, and error analysis;
- **API design:** clear input/output contracts, validation, error handling, determinism, and reliable connection between the panel and the final NER pipeline.

The user interface, code quality, documentation, and responsible data-use discussion support these three areas and may be considered when distinguishing otherwise similar submissions.

Extension challenges

For additional credit, students may:

- highlight entity spans directly in the article text;
- support multiple languages and compare model performance;
- normalize equivalent entities while preserving the original text;
- add confidence scores with an explanation of their meaning;
- containerize the service;
- compare two NER models on the same evaluation set;
- add a batch endpoint for multiple articles.

Definition of done

The assignment is complete when a user can start the application, enter an article title and body, submit the form, and see the identified entities without manually editing code. Another person following the README should be able to reproduce the result and run the tests.